

Working with Concept Sets in Capr

Contents

0.1	Defining a concept set	1
0.2	Including Descendants	1
0.3	Toggling other logic	2
0.4	Fill in missing concept set details	2

```
library(Capr)
```

Capr provides a simple interface for building concept sets from OMOP concept IDs — no database connection required. You can find descendants, exclude concepts, and include mapped terms directly in R. This vignette demonstrates how the concept set API works.

0.1 Defining a concept set

The `cs` function collects integer OMOP concept IDs into a set. The idea is for the function to look like `c()` which in R is use to formulate vectors. Say we found several ingredients that are all ace inhibitors. We can combine those OMOP concept ids into a set as follows:

```
ace1 <- cs(1335471, #benazepril
          1340128, #captopril
          1341927, #enalapril
          1308216, #lisinopril
          1363749, #fosinopril
          name = "aceInhibitors")

ace1
#> -- <Capr Concept Set> aceInhibitors -----
#> # A tibble: 5 x 9
#>   conceptId conceptCode conceptName domainId vocabularyId standardConcept
#>   <int>    <chr>      <chr>      <chr>    <chr>          <chr>
#> 1  1335471 ""         ""         ""      ""           ""
#> 2  1340128 ""         ""         ""      ""           ""
#> 3  1341927 ""         ""         ""      ""           ""
#> 4  1308216 ""         ""         ""      ""           ""
#> 5  1363749 ""         ""         ""      ""           ""
#> # i 3 more variables: includeDescendants <lgl>, isExcluded <lgl>,
#> #   includeMapped <lgl>
```

0.2 Including Descendants

If you were only add the OMOP id for the ingredient, our query would only look at whether that specific ID is among the drug concepts for a set of patients. With OMOP, we often don't want just the ingredient ID but also its descendants. Descendants are child concepts that map to the adult concepts. For example a brand name or specific dosage of lisinopril would have a different OMOP concept Id but it would map to the ingredient term via the vocabulary hierarchy. This helps us include all variations of linisopril that could be seen in the data. In Capr we can easily add this logic using the `descendants` command

```
ace2 <- cs(descendants(1335471, 1340128, 1341927, 1363749, 1308216),
           name = "aceInhibitors")
ace2
#> -- <Capr Concept Set> aceInhibitors -----
#> # A tibble: 5 x 9
#>   conceptId conceptCode conceptName domainId vocabularyId standardConcept
#>   <int> <chr>      <chr>      <chr>      <chr>      <chr>
#> 1  1335471 ""          ""          ""          ""          ""
#> 2  1340128 ""          ""          ""          ""          ""
#> 3  1341927 ""          ""          ""          ""          ""
#> 4  1363749 ""          ""          ""          ""          ""
#> 5  1308216 ""          ""          ""          ""          ""
#> # i 3 more variables: includeDescendants <lgl>, isExcluded <lgl>,
#> #   includeMapped <lgl>
```

0.3 Toggling other logic

A similar strategy can be used to exclude concepts from a set or include mapped. We can also combine this within the `cs` function, as shown below

```
ace3 <- cs(descendants(1335471, 1340128, 1341927, 1363749), exclude(1308216),
           name = "aceInhibitors")
ace3
#> -- <Capr Concept Set> aceInhibitors -----
#> # A tibble: 5 x 9
#>   conceptId conceptCode conceptName domainId vocabularyId standardConcept
#>   <int> <chr>      <chr>      <chr>      <chr>      <chr>
#> 1  1335471 ""          ""          ""          ""          ""
#> 2  1340128 ""          ""          ""          ""          ""
#> 3  1341927 ""          ""          ""          ""          ""
#> 4  1363749 ""          ""          ""          ""          ""
#> 5  1308216 ""          ""          ""          ""          ""
#> # i 3 more variables: includeDescendants <lgl>, isExcluded <lgl>,
#> #   includeMapped <lgl>
```

0.4 Fill in missing concept set details

By default, `Capr` puts in minimal information for the concept set, the concept id. However it is often helpful to fill out the remaining information of the concept using the OMOP vocabularies. To fill out the remaining information, a user needs to connect to a OMOP CDM to access the vocabulary information for the concept in the concept set. First we show a regular concept set, and the json it renders.

```
diclofenac <- cs(descendants(1124300), name = "diclofenac")
cat(as.json(diclofenac))
```

Now we add in the CDM connection to get the remaining information:

```
con <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
diclofenac <- getConceptSetDetails(diclofenac, con, vocabularyDatabaseSchema = "main")
cat(as.json(diclofenac))
```

`getConceptSetDetails()` also accepts a whole `Cohort`. It walks the definition and fills in the details for *every* concept set in one call — entry events, attrition rules, censoring events, the drug-exit concept set, and concept-set attributes such as `conditionSourceConcept()`. Each unique concept set is queried from the vocabulary only once, even when the same set is reused across the cohort.

```
cd <- cohort(  
  entry(conditionOccurrence(diclofenac)),  
  exit = exit(endStrategy = drugExit(diclofenac))  
)  
cd <- getConceptSetDetails(cd, con, vocabularyDatabaseSchema = "main")  
writeCohort(cd, "diclofenac_cohort.json")
```